

INF1103:
Programming Fundamentals with DevOps

Project Report

Deadline: 14 Oct 2026, 23:59

P5 - Team 2	
Members:	Learner ID:
Rui Han	2604707
Ti Seng	2601624
Shakira	2604218
Rania	2604249
Haziq	2603056
Aqmar	2601532

Link to Code Repository: https://github.com/RuiHan-SIT/26INF1103P5-2

Table of Contents

1 Introduction 3
2 Data Flow Diagram 3
3 Data Structure and Storage Design 4
4 Exception Handling Matrix 5
Appendix A: Data storage details 7

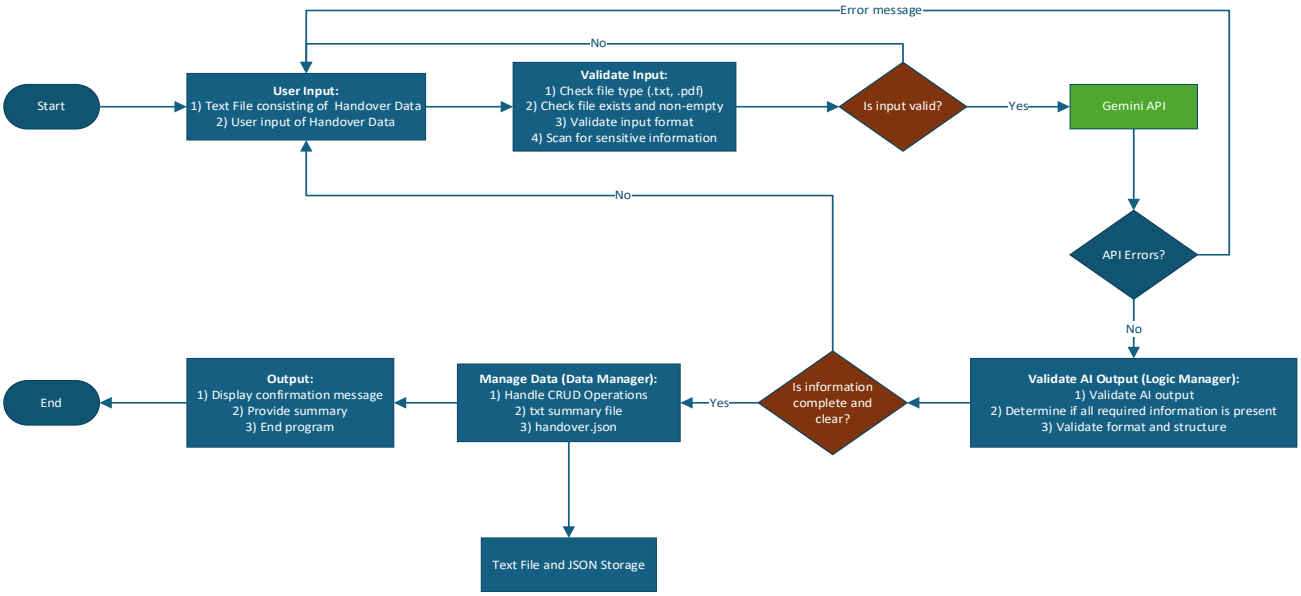
1 Introduction

“Employee handover documents may contain missing or unclear critical information, making it difficult for the receiving employee to understand and continue outstanding work effectively.”

Our project allows an easy handover/takeover process when an employee resigns, goes on temporary reassignment, or takes extended leave where they may need to prepare a handover document for the person taking over their responsibilities. These handovers may be incomplete or poorly structured, causing important details such as outstanding tasks, deadlines, responsible persons, or relevant resources to be missed.

Our project aims to develop an LLM-assisted Employee Handover and Takeover System that simplifies and improves the handover process. The system allows user to submit handover information through direct text input or file uploads. The submitted information undergoes input validation before being passed to the Large Language Model (LLM) for processing. By using the LLM, the system aims to organise handover information into clearer and more structured format which helps employees understand their responsibilities and ensuring smoother transition of work. It also helps to identify missing or unclear details and prompt user to provide the necessary information.

2 Data Flow Diagram



3 Data Structure and Storage Design

3.1 Files utilised by the application

The application stores everything as flat files on the local data/ folder. Notably, there is no database server. All paths are defined once in src/utils/config.py -- (DATA_DIR , HANDOVERS_DIR, SUMMARIES_DIR, & DEFAULT_DATA_FILE), thus every module reads and writes the same locations. A separate file, data/employees.json, acts as a read-only employee registry that confirms each user's name and department from their 7-digit employee ID.

3.2 General Overview of the file types

Refer to Table 1 in Appendix A.

The folder also consists of 8 test inputs (test_01_complete.txt to test_08_mixed_tasks.txt) and README_expected_results.txt, which lists the expected status for each case.

3.3 Data Formats (JSON, TXT)

Effectively, the program uses TXT wherever a user writes or reads, and JSON wherever the program must find, check or update data. TXT keeps notes and summaries readable in any editor; JSON suits nested handover data and matches the AI's output; hence no conversion is needed.

A stored record is derived in 3 steps. Gemini's AI response is sanitised and validated against a Pydantic schema, the logic manager requires an owner and a valid future deadline for each outstanding task, prompting the user for any gaps, and the data manager adds the employee's registered details, a record ID, the saved date and the summary's filename. Consequently, only complete handovers are saved along with a summary text file named by date, employee and time so none is overwritten. Each time the handover is viewed, a viewing history is appended to its summary file.

3.4 Structure of stored records

Crucially, handovers.json holds a list of records, one per handover (Table 2, Appendix A). Each record stores who handed over, the tasks and notes handed over, the linked summary filename, and a viewing history (viewed_by) that gains one entry each time a colleague views it. The various task fields match the AI schema, thus validated output is saved without conversion.

3.5 Effectiveness and Feasibility

Flat JSON and TXT files complement the program as its data is small, document-shaped and used by one user at a time.

Effectiveness: Undoubtedly, each handover is self-contained, mapping directly onto one JSON record with no tables. Nonetheless, the record mirrors the validated AI schema, data is validated once and stored unchanged, hence maintaining consistency. Only complete handovers are saved, and a corrupted file is backed up instead of crashing the program, and deleting a handover also removes its summary file. The JSON file allows the program to find a handover by employee ID, while summary text file gives the next employee an easy-to-read copy.
Feasibility: This functional design requires only Python's JSON library and Pydantic, runs the same on Windows and in Docker, and requires no database setup. A handover is a few kilobytes, so even hundreds of records loads almost instantly.

4 Exception Handling Matrix

Stage	Scenario	Possible Cause	Detection Method	Handling Strategy	Impact on users
Input	Invalid employee ID *	Not 7 digits; not in employees.json; same ID as the person handing over	validate_employee_id(); get_employee() returns None	ID re-prompted	Re-prompted until valid
Input	User file is invalid	1. Wrong file type; 2. File not found; 3. Empty file; 4. Over 5 MB; *5. Not UTF-8; 6. Path outside data/handovers/*	1, 3, 4. ValueError; 2. FileNotFoundError; 5. UnicodeDecodeError; 6. Path check	Error logged; user prompted to retry	Notified and must rectify
Input	Content is only whitespace	Text such as \n or \t only	Logged by logger	Returns None; input requested again	Notified and must rectify
Input	Sensitive data detected *	NRIC/FIN, passport, email, "password"	Regex, scrubadub and keyword check	Blocked before reaching the AI	Must remove data and resubmit
AI	API errors	1. Bad request (400); 2. Bad API key (401/403); 3. Model not found (404); 4. Rate limit (429); 5. Service down (5xx); 6. Network down; 7. Unhandled	Gemini error codes; ServerError; TimeoutError/ConnectionError; fail-safe catch	User notified and asked for action; 429, 5xx and timeouts first retried up to 5 times automatically	Notified of service unavailability
AI	LLM output malformed	1. Output wrapped in code fences or quotes; 2. Malformed JSON	ValueError	Output cleaned and repaired; user alerted if it still fails	Alerted; cannot proceed
AI	Pydantic validation error	Output does not match the prompted schema	ValidationError	User alerted	Alerted; cannot proceed

Stage	Scenario	Possible Cause	Detection Method	Handling Strategy	Impact on users
Validation	Incomplete handover *	Task name, description or owner missing; no tasks found	Logic manager validation errors	Missing fields re-prompted; resubmit if no tasks	Fills in only the gaps
Validation	Invalid deadline	Deadline does not meet business rules	Logic manager checks output after Pydantic	Handled via ValueError/TypeError; re-prompted	Notified and must rectify
Storage	Cannot load handovers.json	Corrupted file; locked by another process	OSError on open/read	Error logged and shown	Alerted; cannot proceed
Storage	Cannot save handovers.json *	Disk full; no write permission	OSError in save_records()	Warning shown	Handover not stored
Storage	Cannot load employees.json	Corrupted file; locked by another process	JSONDecodeError on read	Error logged and shown	Cannot be verified by employee ID
Output	Cannot generate summary.txt	Read-only folder; disk full	OSError on write	Clean error message	No summary file generated
Output	Cannot append viewing data	Read-only file; disk full	OSError on append	Clean error message	Summary lacks latest viewing data

Appendix A: Data storage details

File/folder	Format	Written by	Read by
data/handovers/*.txt	Plain text, UTF-8	The employee handing over (outside the app)	input_handler.py
data/handovers.json	JSON array	data_manager.py	data_manager.py
data/employees.json	JSON object keyed by employee ID	Maintained manually (committed to Git)	data_manager.py
data/summaries/<date>_<name>_<id>_<time>.txt	Formatted plain text	output.py	The human reader
data/handovers.json.corrupted.bak	Raw text copy	data_manager.py	The developer
.env	Key=value text	The developer	gemini_client.py

Table 1: Files used by the application

Key	Type	Source
id	integer	add_record(): highest id + 1
saved_date	string, YYYY-MM-DD	add_record()
name	string	employees.json registry
employee_id	string, 7 digits	User input, checked against the registry
department	string	employees.json registry
filename	string	Summary .txt written by output.py
outstanding_tasks	list of objects	AI + logic manager, sorted by deadline
outstanding_tasks[].task	string	AI / user fix-up
outstanding_tasks[].description	string	AI / user fix-up
outstanding_tasks[].owners	list of strings	AI / user fix-up
outstanding_tasks[].deadline	string, DD Mon YYYY	AI, validated and reformatted
bau_tasks	list of objects (task, description, owners)	AI / user fix-up

Key	Type	Source
id	integer	add_record(): highest id + 1
saved_date	string, YYYY-MM-DD	add_record()
name	string	employees.json registry
employee_id	string, 7 digits	User input, checked against the registry

Table 2: Structure of a stored handover record